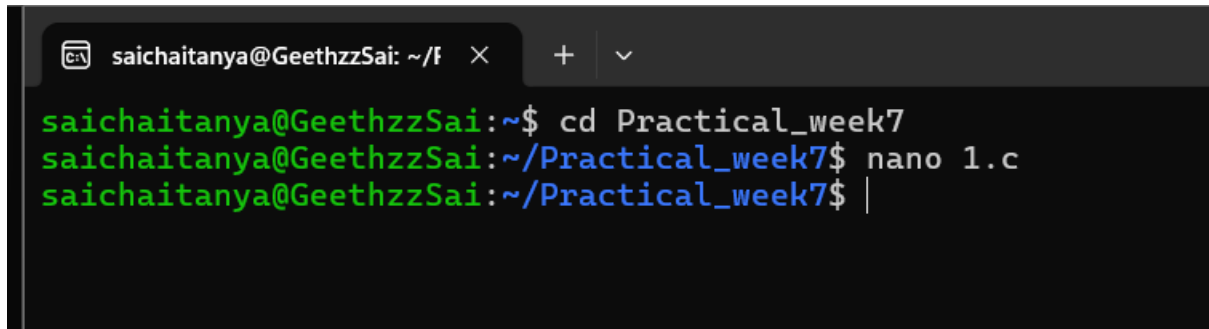


PRACTICAL WEEK 7

Task 1: Write a program that prints the addresses Of code, global, static, heap, and stack variables. Analyze the Linux process address space layout and explain each memory segment.

1) Create a file

A terminal window with a dark background. The title bar shows 'saichaitanya@GeethzzSai: ~/f' and a close button. The terminal content shows three lines of commands and their outputs: the first line is 'cd Practical_week7' resulting in the prompt 'saichaitanya@GeethzzSai:~/Practical_week7\$'; the second line is 'nano 1.c' resulting in the prompt 'saichaitanya@GeethzzSai:~/Practical_week7\$'; and the third line is a vertical bar '|' resulting in the prompt 'saichaitanya@GeethzzSai:~/Practical_week7\$ |'.

Then press enter to open command

2) write a code

```
GNU nano 8.7.1 1.c
#include <stdio.h>
#include <stdlib.h>

int global_var = 10;           // Global variable
int global_uninit;            // Global uninitialized variable

void code_function()
{
    printf("Address of code function : %p\n", (void *)code_function);
}

int main()
{
    static int static_var = 20; // Static variable
    int stack_var = 30;         // Stack variable

    int *heap_var = malloc(sizeof(int));
    *heap_var = 40;

    printf("Linux Process Memory Address Layout\n\n");

    code_function();

    printf("Address of global variable      : %p\n", (void *)&global_var);
    printf("Address of global uninitialized : %p\n", (void *)&global_uninit);
    printf("Address of static variable      : %p\n", (void *)&static_var);
    printf("Address of heap variable         : %p\n", (void *)heap_var);
    printf("Address of stack variable         : %p\n", (void *)&stack_var);

    free(heap_var);

    return 0;
}
```

- 1) To save press control O
- 2) To come back to shell press control X
- 3) Then DO compilation

```
saichaitanya@GeethzzSai: ~/f  × + v
saichaitanya@GeethzzSai:~$ cd Practical_week7
saichaitanya@GeethzzSai:~/Practical_week7$ nano 1.c
saichaitanya@GeethzzSai:~/Practical_week7$ gcc 1.c -o 1
saichaitanya@GeethzzSai:~/Practical_week7$ |
```

OUTPUT:

```
saichaitanya@GeethzzSai: ~/f × + v
saichaitanya@GeethzzSai:~$ cd Practical_week7
saichaitanya@GeethzzSai:~/Practical_week7$ nano 1.c
saichaitanya@GeethzzSai:~/Practical_week7$ gcc 1.c -o 1
saichaitanya@GeethzzSai:~/Practical_week7$ ./1
Linux Process Memory Address Layout

Address of code function : 0x6056213561c9
Address of global variable      : 0x605621359010
Address of global uninitialized : 0x60562135901c
Address of static variable      : 0x605621359014
Address of heap variable        : 0x6056588e1010
Address of stack variable       : 0x7fff5cef318c
saichaitanya@GeethzzSai:~/Practical_week7$ |
```

Task-2: Make use of the `/proc//maps` file and memory analysis tools to study the memory organization of a running process. Prepare a report explaining virtual memory mappings.

```
saichaitanya@GeethzzSai:~/Practical_week7$  
saichaitanya@GeethzzSai:~/Practical_week7$  
saichaitanya@GeethzzSai:~/Practical_week7$ nano 2.c|
```

```
GNU nano 8.7.1 2.c *  
#include <stdio.h>  
#include <stdlib.h>  
#include <unistd.h>  
  
int global_var = 10;  
int global_uninit;  
  
int main()  
{  
    static int static_var = 20;  
    int stack_var = 30;  
  
    int *heap_var = malloc(sizeof(int));  
    *heap_var = 40;  
  
    printf("Process ID: %d\n", getpid());  
    printf("Code : %p\n", (void *)main);  
    printf("Global : %p\n", (void *)&global_var);  
    printf("Static : %p\n", (void *)&static_var);  
    printf("Heap : %p\n", (void *)heap_var);  
    printf("Stack : %p\n", (void *)&stack_var);  
  
    printf("\nProcess is running...\n");  
  
    while (1)  
    {  
        sleep(1);  
    }  
  
    return 0;  
}
```

OUTPUT:

```
saichaitanya@GeethzzSai:~/Practical_week7$
saichaitanya@GeethzzSai:~/Practical_week7$ nano 2.c
saichaitanya@GeethzzSai:~/Practical_week7$ gcc 2.c -o 2
saichaitanya@GeethzzSai:~/Practical_week7$ ./2
Process ID: 845
Code   : 0x5c8d5ba951c9
Global : 0x5c8d5ba98010
Static : 0x5c8d5ba98014
Heap   : 0x5c8d5ee44010
Stack  : 0x7ffffebc52aac

Process is running...
```

```
saichaitanya@GeethzzSai: ~/Practical_week7$ cat /proc/845/maps
5c8d5ba94000-5c8d5ba95000 r--p 00000000 08:30 52147 /home/saichaitanya/Practical_week7/2
5c8d5ba95000-5c8d5ba96000 r-xp 00001000 08:30 52147 /home/saichaitanya/Practical_week7/2
5c8d5ba96000-5c8d5ba97000 r--p 00002000 08:30 52147 /home/saichaitanya/Practical_week7/2
5c8d5ba97000-5c8d5ba98000 r--p 00002000 08:30 52147 /home/saichaitanya/Practical_week7/2
5c8d5ba98000-5c8d5ba99000 rw-p 00003000 08:30 52147 /home/saichaitanya/Practical_week7/2
5c8d5ee44000-5c8d5ee65000 rw-p 00000000 00:00 0 [heap]
7e80eae00000-7e80eae28000 r--p 00000000 08:30 13647 /usr/lib/x86_64-linux-gnu/libc.so.6
7e80eae28000-7e80eafc0000 r-xp 00028000 08:30 13647 /usr/lib/x86_64-linux-gnu/libc.so.6
7e80eafc0000-7e80eb00e000 r--p 001c0000 08:30 13647 /usr/lib/x86_64-linux-gnu/libc.so.6
7e80eb00e000-7e80eb012000 r--p 002d0000 08:30 13647 /usr/lib/x86_64-linux-gnu/libc.so.6
7e80eb012000-7e80eb014000 rw-p 00211000 08:30 13647 /usr/lib/x86_64-linux-gnu/libc.so.6
7e80eb014000-7e80eb021000 rw-p 00000000 00:00 0
7e80eb11d000-7e80eb120000 rw-p 00000000 00:00 0
7e80eb126000-7e80eb128000 rw-p 00000000 00:00 0
7e80eb128000-7e80eb12c000 r--p 00000000 00:00 0
7e80eb12c000-7e80eb12e000 r--p 00000000 00:00 0
7e80eb12e000-7e80eb130000 r-xp 00000000 00:00 0
7e80eb130000-7e80eb131000 r--p 00000000 08:30 13644 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7e80eb131000-7e80eb160000 r-xp 00001000 08:30 13644 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7e80eb160000-7e80eb16b000 r--p 00030000 08:30 13644 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7e80eb16b000-7e80eb16d000 r--p 0003b000 08:30 13644 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7e80eb16d000-7e80eb16e000 rw-p 0003d000 08:30 13644 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7e80eb16e000-7e80eb16f000 rw-p 00000000 00:00 0
7ffffebc33000-7ffffebc54000 rw-p 00000000 00:00 0 [stack]
saichaitanya@GeethzzSai:~/Practical_week7$ |
```